



北京師範大學
BEIJING NORMAL UNIVERSITY

《计算机图形学》课程实验

实验三：摄像机 (参考操作教程)

授课教师：张鸿文

助教：李玉慧

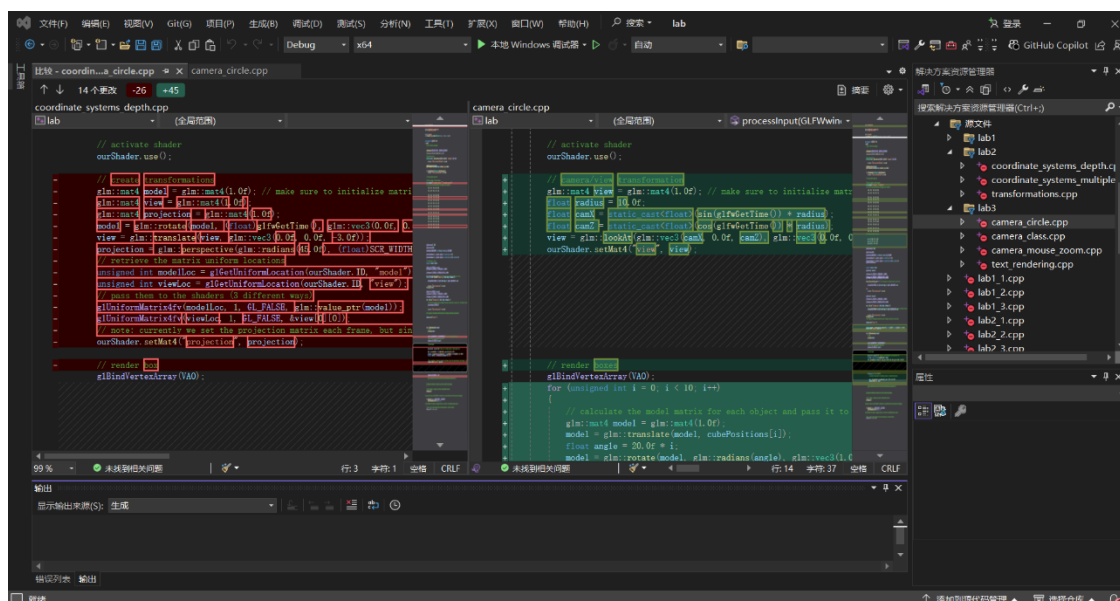
课程网页：<https://zhanghongwen.cn/cg>

计算机图形学实验三——观察变换：摄像机

1. 阅读“摄像机”一节中的“摄像机”部分，运行示例程序“camera_circle.cpp”

https://learnopengl.com/code_viewer_gh.php?code=src/1.getting_started/7.1.camera_circle/camera_circle.cpp

将“camera_circle.cpp”与“coordinate_systems_depth.cpp”对比，主要区别有：将固定不变的透视投影矩阵 projection 提取到循环外，调用 glm 的 LookAt 函数为观察矩阵 view 设置摄像机相关参数，为十个不同位置的立方体设置不同的模型矩阵 model。



2. 在实验二的基础上，从不同摄像机角度观察以设置深度测试的立方体。

在实验二中，我们将任务一的着色器中的 transform 矩阵复用到任务二，但也可以在着色器外部对 model 变换，表示单个物体的变换：

```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
"out vec2 TexCoord;\n"
"uniform mat4 model;\n"
"uniform mat4 view;\n"
"uniform mat4 projection;\n"

"void main() \n"
"{\n"
"    gl_Position = projection * view * model * vec4(aPos, 1.0f);\n"
"    ourColor = aColor;\n"
"    TexCoord = vec2(aTexCoord.x, 1.0 - aTexCoord.y);\n"
"}\n";
```

删去 transform 矩阵传递相关的函数调用，并设置摄像机相关的变换矩阵，投影矩阵 projection 不变，观察矩阵 view 随时间变换，模型矩阵 model 不更改模型本身的位置方向。

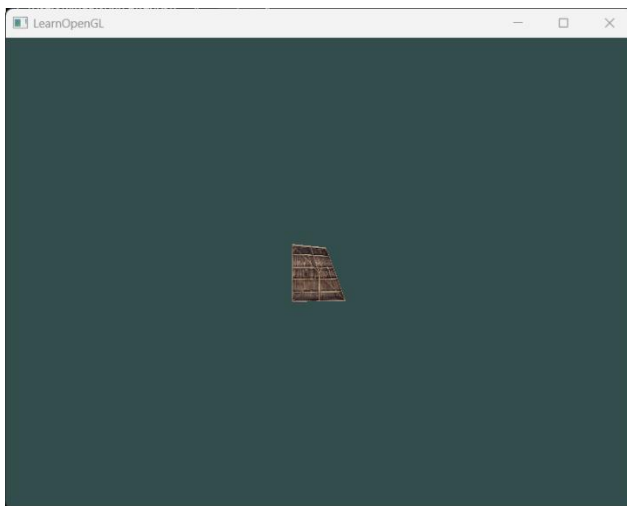
在观察矩阵中 LookAt 的参数中，第一个参数为摄像机位置，设置 y 方向即上下位置不变，x、z 分别为 sin、cos，随时间绕圆心旋转，半径设置为 10.0；第二个参数为摄像机目标，设置为指向中心位置；第三个参数为摄像机上轴，设置为指向世界坐标系中的上方：

```
// projection
glm::mat4 projection = glm::mat4(1.0f);
projection = glm::perspective(glm::radians(45.0f), (float)SCR_WIDTH /
(float)SCR_HEIGHT, 0.1f, 100.0f);

// camera/view
glm::mat4 view = glm::mat4(1.0f); // make sure to initialize matrix to identity
matrix first
float radius = 10.0f;
float camX = static_cast<float>(sin glfwGetTime()) * radius;
float camZ = static_cast<float>(cos glfwGetTime()) * radius;
view = glm::lookAt(glm::vec3(camX, 0.0f, camZ), glm::vec3(0.0f, 0.0f, 0.0f),
glm::vec3(0.0f, 1.0f, 0.0f));

// model
glm::mat4 model = glm::mat4(1.0f);
```

运行结果如下，物体不断旋转，由于摄像头的旋转半径设置为 10.0，距离物体较远，物体渲染后较小：

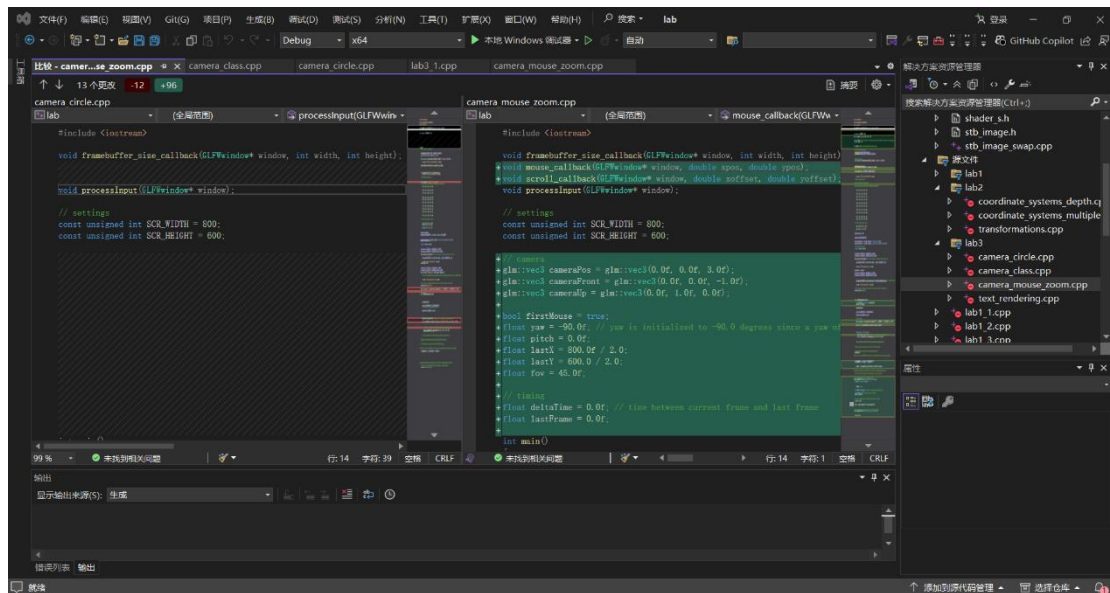


3. 阅读“摄像机”一节中的“自由移动”与“视角移动”部分，运行示例程序“camera_mouse_zoom.cpp”

https://learnopengl.com/code_viewer_gh.php?code=src/1.getting_started/7.3.camera_mouse_zoom/camera_mouse_zoom.cpp

将“camera_mouse_zoom.cpp”与“camera_circle.cpp”对比，主要区别有：增加了鼠标与键盘的回调函数，增加了摄像机参数的全局变量以便根据键鼠反馈修改，仍将投影矩阵 projection 放在循环内以根据键鼠修改后的全局变量重新设置 fov，在循环开始时增加了与

上一帧的时间差。



4. 在任务一的基础上，使用键鼠控制摄像头来观察立方体

键盘输入的 processInput 函数在“你好，窗口”一节中的“输入”部分就进行了介绍，通过 glfwGetKey 获取键盘调用信息；鼠标输入的 mouse_callback 与 scroll_callback 也与示例代码一致，自行添加；注意要调用 glfwSetCursorPosCallback 和 glfwSetScrollCallback 来设置回调函数。

鼠标控制根据与上一时刻的位置差来确定 cameraFront，作为 LookAt 的第二个参数，在当前位置的基础上设置摄像机目标；俯仰角 pitch、偏横角 yaw、横滚角 roll 值的定义见“欧拉角”部分，是以摄像机坐标系分别绕其 x 轴、y 轴、z 轴旋转角度的值。

我们暂时不需要绕 z 轴旋转，也不需要更改 y 轴方向，cameraPos 默认设置为正上方向，因此不设置 roll。

```
glm::vec3 cameraFront = glm::vec3(0.0f, 0.0f, -1.0f);

int main()
...
while (!glfwWindowShouldClose(window))
...

    glm::mat4 view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);

void mouse_callback(GLFWwindow* window, double xposIn, double yposIn)
...

    glm::vec3 front;
    front.x = cos(glm::radians(yaw)) * cos(glm::radians(pitch));
    front.y = sin(glm::radians(pitch));
    front.z = sin(glm::radians(yaw)) * cos(glm::radians(pitch));
    cameraFront = glm::normalize(front);
```

键盘控制中的前后左右即 cameraPos 根据 GLFW_KEY_W (键盘“W”键)、GLFW_KEY_S (键

盘“S”键)、GLFW_KEY_A (键盘“A”键)、GLFW_KEY_D (键盘“D”键) 状态修改, cameraPos 将作为 LookAt 的第一个参数, 直接修改摄像机位置; 其中 W 键和 S 键根据当前摄像机朝向直接修改位置, A 键和 D 键对应的左右方向 (x 轴) 需要根据摄像机方向 (z 轴) 和上轴 (y 轴) 叉乘计算。。

```
glm::vec3 cameraPos = glm::vec3(0.0f, 0.0f, 3.0f);

int main()
...
while (!glfwWindowShouldClose(window))
...
    glm::mat4 view = glm::lookAt(cameraPos, cameraPos + cameraFront, cameraUp);

void processInput(GLFWwindow* window)
...
if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
    cameraPos += cameraSpeed * cameraFront;
if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
    cameraPos -= cameraSpeed * cameraFront;
if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
    cameraPos -= glm::normalize(glm::cross(cameraFront, cameraUp)) * cameraSpeed;
if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
    cameraPos += glm::normalize(glm::cross(cameraFront, cameraUp)) * cameraSpeed;
```

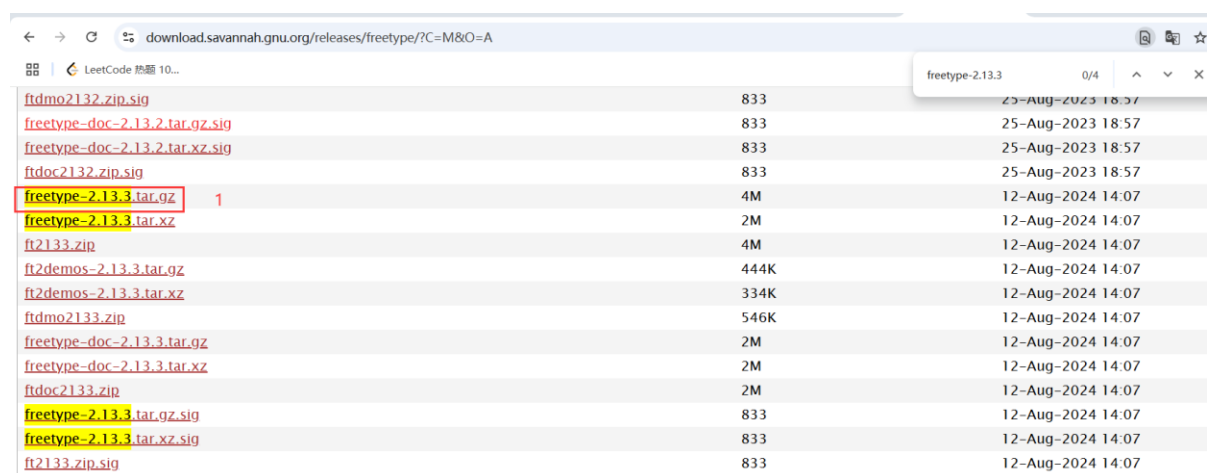
—————下述为附加题部分, 建议学有余力的同学完成, 不强制要求—————

5. 安装 FreeType 库

NuGet 程序包管理器中的 FreeType 库为旧版, 且不会自动配置 lib 库和 dll 库, 因此我们手动安装。如果之前通过 NuGet 安装失败, 需要先卸载。

<https://download.savannah.gnu.org/releases/freetype/>

选择最新的“freetype-2.13.3.tar.gz”下载并解压



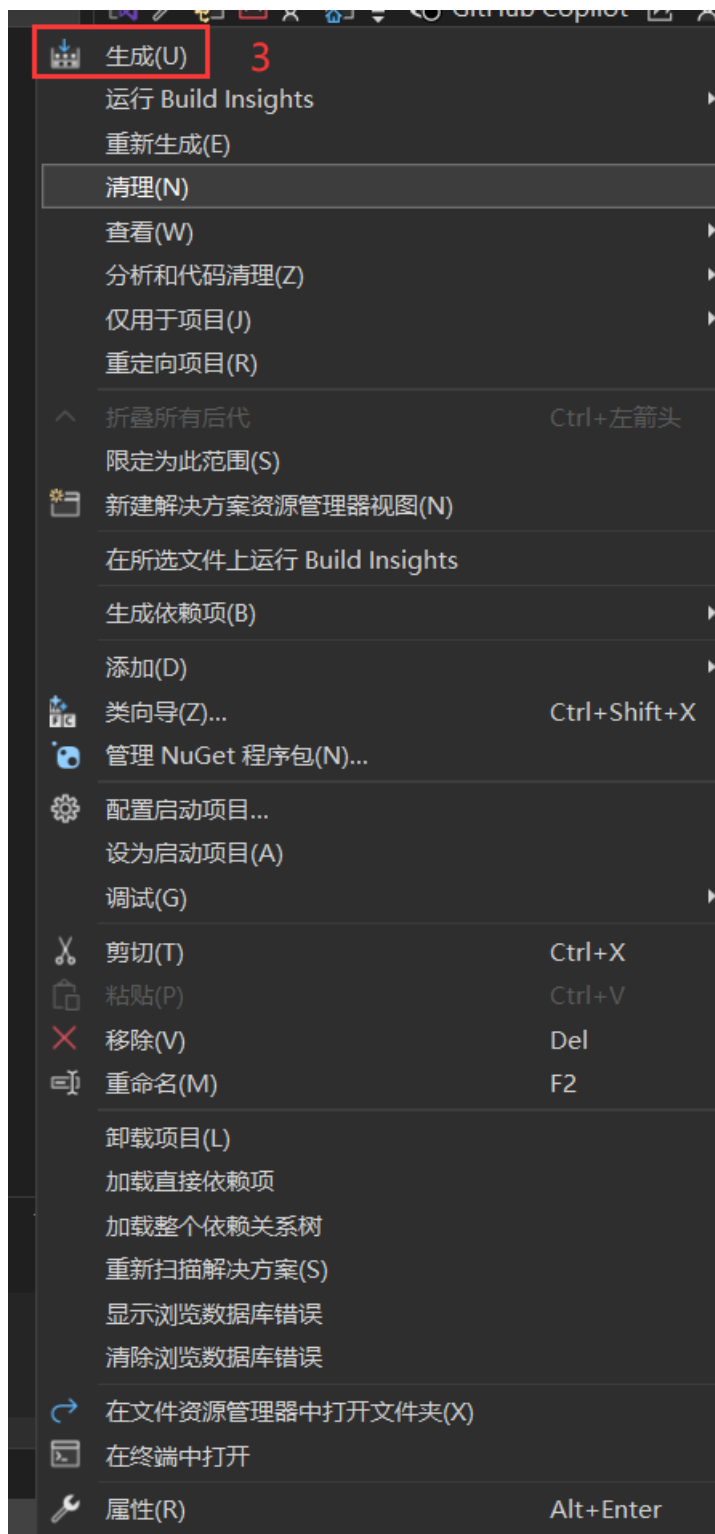
ftdemo2132.zip.sig	833	25-Aug-2023 18:57
freetype-doc-2.13.2.tar.gz.sig	833	25-Aug-2023 18:57
freetype-doc-2.13.2.tar.xz.sig	833	25-Aug-2023 18:57
ftdoc2132.zip.sig	833	25-Aug-2023 18:57
freetype-2.13.3.tar.gz	4M	12-Aug-2024 14:07
freetype-2.13.3.tar.xz	2M	12-Aug-2024 14:07
ft2133.zip	4M	12-Aug-2024 14:07
ft2demos-2.13.3.tar.gz	444K	12-Aug-2024 14:07
ft2demos-2.13.3.tar.xz	334K	12-Aug-2024 14:07
ftdemo2133.zip	546K	12-Aug-2024 14:07
freetype-doc-2.13.3.tar.gz	2M	12-Aug-2024 14:07
freetype-doc-2.13.3.tar.xz	2M	12-Aug-2024 14:07
ftdoc2133.zip	2M	12-Aug-2024 14:07
freetype-2.13.3.tar.gz.sig	833	12-Aug-2024 14:07
freetype-2.13.3.tar.xz.sig	833	12-Aug-2024 14:07
ft2133.zip.sig	833	12-Aug-2024 14:07

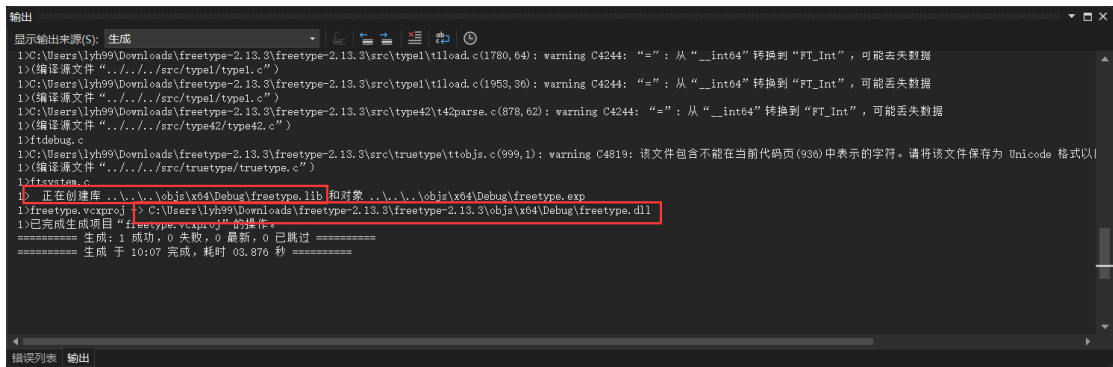
双击“MSBuild.sln”, 在 Visual Studio 打开 freetype 的项目

🔄 > 下载 > freetype-2.13.3 > freetype-2.13.3 >				
📁 📄 🔍 🔄 🗑️ ⬆️ 排序 ▾ ☰ 查看 ▾ ⋮				
名称	修改日期	类型	大小	
📄 vms_make.com	2024/7/18 15:14	MS-DOS 应用程序	63 KB	
📄 .clang-format	2024/7/18 15:14	CLANG-FORMA...	1 KB	
📄 autogen.sh	2024/7/18 15:14	SH 源文件	5 KB	
📄 configure	2024/7/18 15:14	文件	4 KB	
📄 LICENSE.TXT	2024/7/18 15:14	文本文档	3 KB	
📄 Makefile	2024/7/18 15:14	文件	1 KB	
📄 MSBuild.rsp	2024/7/18 15:14	RSP 文件	1 KB	
📄 MSBuild.sln	2024/7/18 15:14	Visual Studio Sol...	4 KB	
📄 README.git	2024/7/18 15:14	GIT 文件	4 KB	
📁 docs	2024/8/12 19:00	文件夹		
📁 subprojects	2024/8/12 19:00	文件夹		

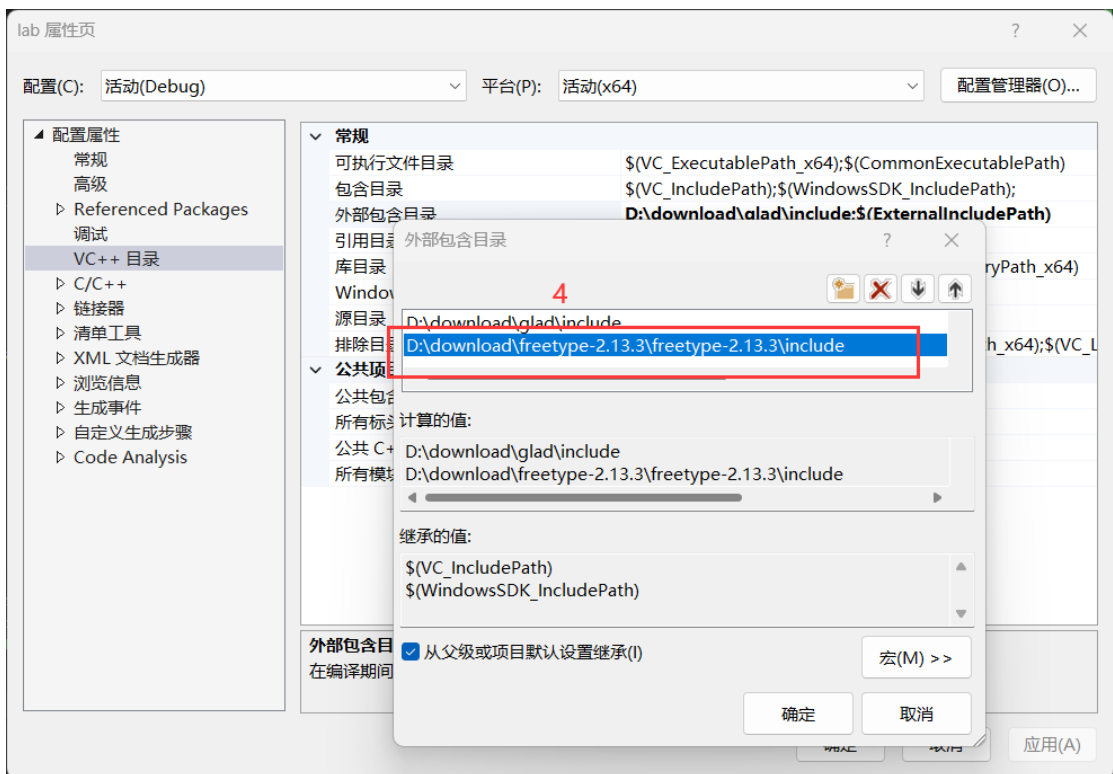
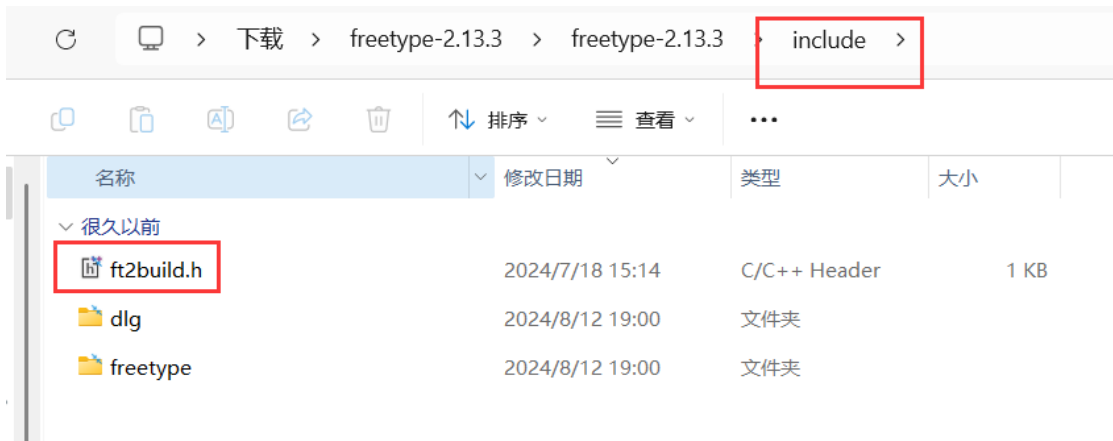
2

“解决方案资源管理器”-“MSBuild”-（右键单击）-“生成”，不需要运行。在输出中可以看到生成的 dll 文件的路径为“freetype-2.13.3\objs\x64\Debug\fontconfig.dll”，生成的 lib 文件也在该路径下；注意生成的路径和配置有关，我们此时的配置为“x64”和“Debug”，该配置也需要与实验项目相符。



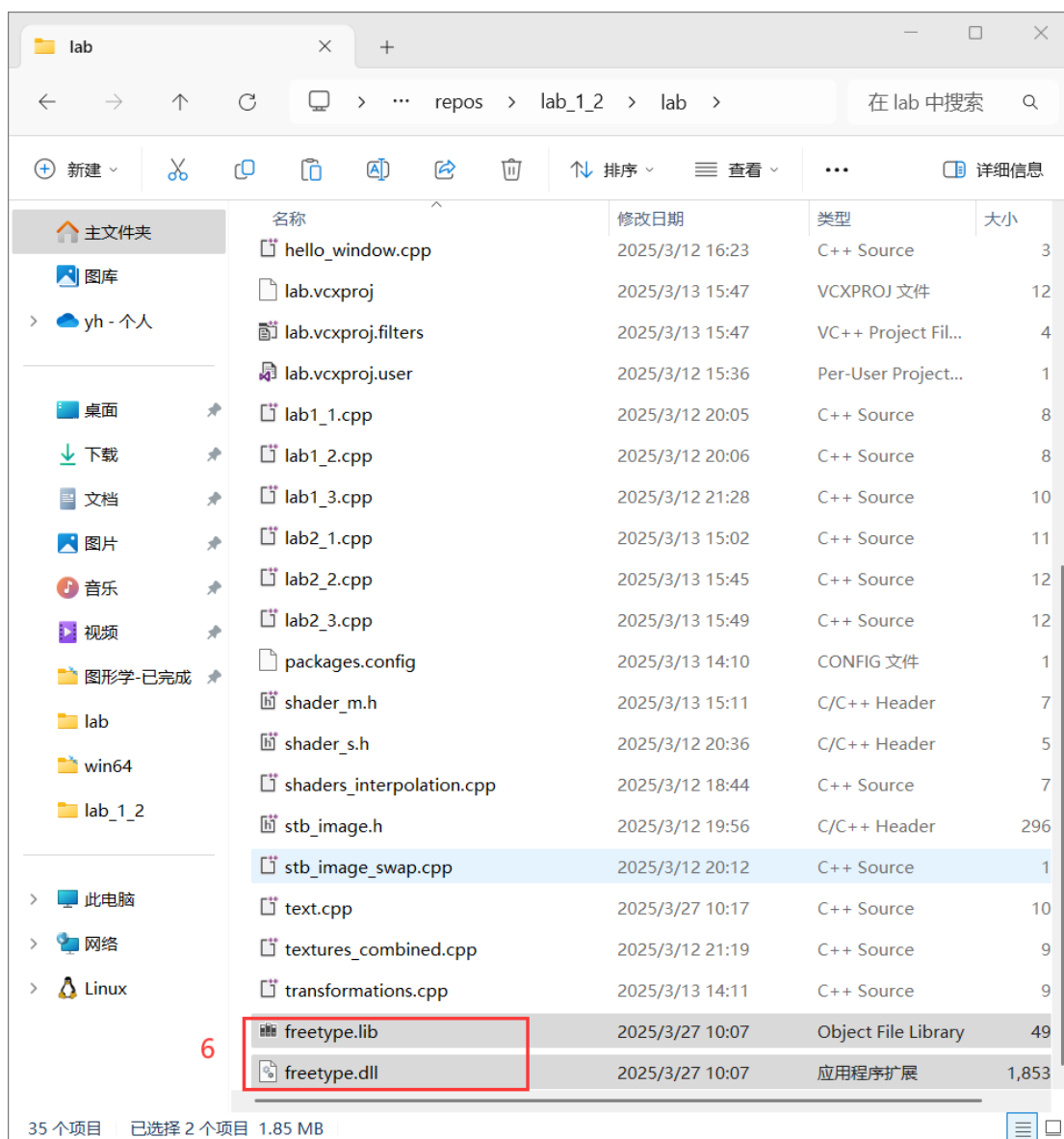


与 glad/glad.h 相同，将 include 路径添加到附加包含目录中，以将 ft2build.h 添加到项目。



将之前生成的“freetype.dll”和“freetype.lib”复制到实验项目的文件夹中，与其他代码文件在同一目录下。通过“解决方案资源管理器”-“项目名”-（右键单击）-“在文件资源管理器中打开文件夹”可以直接打开项目对应的文件夹。

🔄 > 🖥️ > 下载 > freetype-2.13.3 > freetype-2.13.3 > objs > x64 > Debug >				
📁 📄 📖 🗑️ ⬆️ 排序 ▾ ≡ 查看 ▾ ⋮				
名称	修改日期	类型	大小	
▼ 今天				
📄 freetype.dll	2025/3/27 10:07	应用程序扩展	1,853 KB	
📄 freetype.dll.recipe	2025/3/27 10:07	RECIPE 文件	1 KB	
⚙️ freetype.exp	2025/3/27 10:07	Exports Library F...	29 KB	
🔗 freetype.ilc	2025/3/27 10:07	Incremental Link...	1,131 KB	
📦 freetype.lib	2025/3/27 10:07	Object File Library	49 KB	
📄 freetype.log	2025/3/27 10:07	文本文档	5 KB	
🔗 freetype.pdb	2025/3/27 10:07	Program Debug ...	2,908 KB	
📄 freetype.vcxproj.FileListAbsolute.txt	2025/3/27 10:07	文本文档	1 KB	
🔗 ftdebug.obj	2025/3/27 10:07	Object File	40 KB	



在需要调用 freetype 的开头，加上一行代码，以链接到 lib 文件

```
#pragma comment(lib, "freetype.lib")
```

6. 阅读“实战”章节中的“文本渲染”一节，运行示例程序“text_rendering.cpp”

中文版教程仍为旧版，直接点击代码链接会跳转到 glew 的示例代码，此处为英文版教程的 glad 链接。

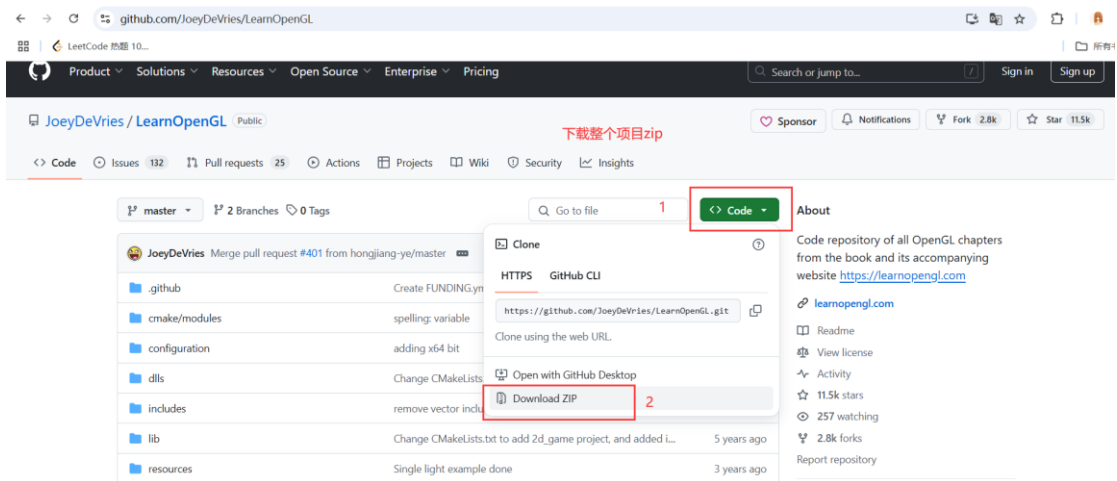
https://learnopengl.com/code_viewer_gh.php?code=src/7.in_practice/2.text_rendering/text_rendering.cpp

shader.h 可以单击网页中带下划线的链接，跳转复制到新建的头文件中。

https://learnopengl.com/code_viewer_gh.php?code=includes/learnopengl/shader.h

字体可以从 github 下载，如果下载单个文件有问题（raw.github 无法访问），也可以下载整个项目文件后，再提取需要的文件。也可以自定义字体，格式为.ttf 即可。

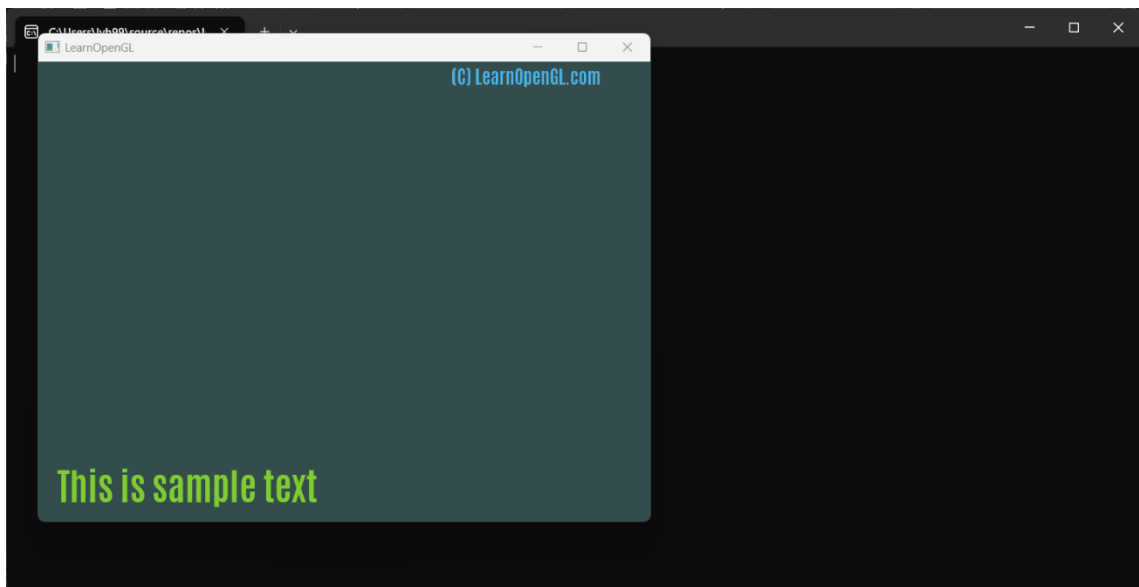
<https://github.com/JoeyDeVries/LearnOpenGL/blob/master/resources/fonts/Antonio-Bold.ttf>



如果在项目新建 shader.h, 还需要将#include <shader.h>改为#include "shader.h"; 除此之外, font_name 一行直接删除 FileSystem 相关函数, 路径改为已下载字体文件的本地路径, 如:

```
std::string font_name = "/Antonio-Bold.ttf";
```

运行结果如下:



将“text_rendering.cpp”与“coordinate_systems_depth.cpp”对比, 改动较多, 但我们只需要提取和文本渲染相关的部分。

7. 在任务二的基础上, 根据不同的键鼠操作, 实时输出当前的摄像头位置、方向、上轴。

文本渲染和物体渲染的代码基本上是独立的, 使用不同的着色器、VAO、VBO、texture; 如果对于不同着色器的用法有疑问, 可以将“你好, 三角形”的练习三参考解答作为示例代码。

在合并过程中, 我们需要对在两个 cpp 文件中重复命名的变量修改变量名, 如 VAO、vertices 等。

除此之外, 由于在渲染文本前, 设置了剔除背面与颜色混合, 即:

```
glEnable(GL_CULL_FACE);  
glEnable(GL_BLEND);
```

```
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
```

在渲染物品前，需要将设置恢复，以免影响物体渲染：

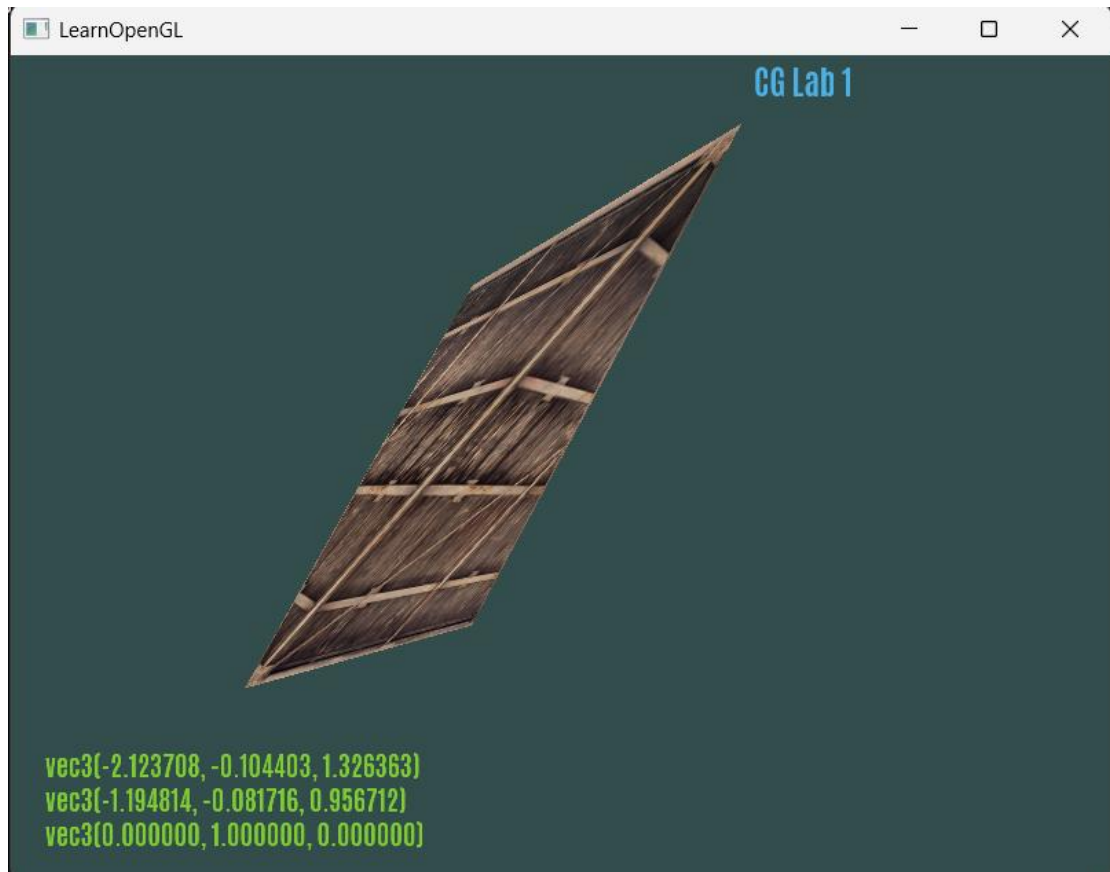
```
glDisable(GL_CULL_FACE);  
glDisable(GL_BLEND);
```

对于 glm 的 mat 矩阵转字符串，我们直接调用 glm::to_string 得到矩阵信息

```
#include <string>  
#define GLM_ENABLE_EXPERIMENTAL  
#include <glm/gtx/string_cast.hpp>  
  
using std::string;
```

```
string cameraPos_str = glm::to_string(cameraPos);
```

最终效果如下：



由于任务 3 较复杂，此处给出参考主体代码。部分简单函数、固定流程、变量（“--pass—”表示）与要求运行的示例代码中的对应内容完全一致，可以在对应章节找到，不在本代码中赘述；建议在自己任务二的代码上结合“text_rendering.cpp”参考修改：

```
#pragma comment(lib, "freetype.lib")  
  
#include <iostream>
```

```
#include <map>
#include <string>
#include <vector>

#include <iostream>
#include <sstream>

#include <glad/glad.h>
#include <GLFW/glfw3.h>

#include "stb_image.h"

#define GLM_ENABLE_EXPERIMENTAL
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
#include <glm/gtx/string_cast.hpp>

#include <ft2build.h>
#include FT_FREETYPE_H

using std::string;
using std::vector;
```

```
// 视口回调函数
void framebuffer_size_callback(GLFWwindow* window, int width, int height);
// 键盘回调函数（键盘实际并没有回调，而是调用glfwGetKey）
void mouse_callback(GLFWwindow* window, double xpos, double ypos);
void scroll_callback(GLFWwindow* window, double xoffset, double yoffset);
void processInput(GLFWwindow* window);

// 将示例代码部分模块进行封装，都是示例代码复制过来到函数里的，不再赘述
unsigned int linkShaderProgram(const char* vertexShaderSource, const char*
fragmentShaderSource);
int initFreeType();
void createTextData();
void createObjectData();
void createObjectTexture();
void loadObjectTexture(unsigned int shaderProgram_object);

// 渲染函数
void RenderText(unsigned int shaderProgram_text, std::string text, float x, float y,
float scale, glm::vec3 color);
vector<string> RenderObject(unsigned int shaderProgram_object);
```

本代码中不使用 shader.h 头文件，而是在单个文件中处理着色器；如果使用 shader 类，请务必清楚你所用的 shader.h 每个函数代表什么内容，如 shaderProgram 等同于 ourShader.ID，ourShader.use() 等同于 glUseProgram(shaderProgram)。

```
/****** 全局常量：固定设置与源码 *****/

// 全局常量，OpenGL窗口设置，无法修改
const unsigned int SCR_WIDTH = 800;
const unsigned int SCR_HEIGHT = 600;

// 全局常量，物体顶点着色器源码
const char* vertexShaderSource_object = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
"out vec2 TexCoord;\n"
"uniform mat4 model;\n"
"uniform mat4 view;\n"
"uniform mat4 projection;\n"

"void main()\n"
"{\n"
"    gl_Position = projection * view * model * vec4(aPos, 1.0f);\n"
"    ourColor = aColor;\n"
"    TexCoord = vec2(aTexCoord.x, 1.0 - aTexCoord.y);\n"
"}\n0";

// 全局常量，物体片段着色器源码
const char* fragmentShaderSource_object = "#version 330 core\n"
"out vec4 FragColor;\n"
"in vec3 ourColor;\n"
"in vec2 TexCoord;\n"
"uniform sampler2D ourTexture;\n"
"void main()\n"
"{\n"
"    FragColor = texture(ourTexture, TexCoord);\n"
"}\n\n0";

// 全局常量，文本顶点着色器源码
const char* vertexShaderSource_text = "#version 330 core\n"
"layout (location = 0) in vec4 vertex;\n"
"out vec2 TexCoords;\n"
"uniform mat4 projection;\n"
```

```

"void main()\n"
"{\n"
"    gl_Position = projection * vec4(vertex.xy, 0.0, 1.0);\n"
"    TexCoords = vertex.zw;\n"
"}\n0";

// 全局常量，文本片段着色器源码
const char* fragmentShaderSource_text = "#version 330 core\n"
"in vec2 TexCoords;\n"
"out vec4 color;\n"
"uniform sampler2D text;\n"
"uniform vec3 textColor;\n"
"void main()\n"
"{\n"
"    vec4 sampled = vec4(1.0, 1.0, 1.0, texture(text, TexCoords).r);\n"
"    color = vec4(textColor, 1.0) * sampled;\n"
"}\n0";

```

```

/***** 全局变量：摄像机参数，键鼠控制 *****/
// 由于鼠标采用回调机制，我们不能以return的方式来得到处理鼠标反馈后的信息，此处使用全局变量获取
// 全局变量，摄像机参数
--pass--
// 全局变量，用于鼠标控制
--pass--
// 全局变量，用于键盘控制
--pass--

/***** 全局变量：字体信息 *****/
--pass--

/***** 全局变量：缓冲信息 *****/
// 物体对象
unsigned int VAO_object, VBO_object, EBO_object; // 物体数组及缓冲对象
unsigned int texture_object; // 物体纹理对象
// 文本对象
unsigned int VAO_text, VBO_text; // 文本数组及缓冲对象

/***** 全局变量，固定物体 *****/
// 顺序：RGB
float vertices[] = {
    0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 1.0f, 1.0f, // top right, blue
    0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, // bottom right, green
    -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f, // bottom left, red

```

```

        -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f // top left -> front
    };

    unsigned int indices[] = { // note that we start from 0!
        0, 1, 2, // first Triangle
        2, 3, 0, // first Triangle
        3, 0, 1, // second Triangle
        1, 2, 3, // second Triangle
    };

```

```

/***** 主函数 *****/

int main()
{
    // glfw: initialize and configure
    --pass--
    // 创建窗口
    --pass--
    // 回调函数
    --pass--
    // 鼠标设置，隐藏光标，注释可显示光标，但影响视角范围
    // glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);

    // glad: load all OpenGL function pointers
    // -----
    --pass--

    glEnable(GL_DEPTH_TEST);

    // 物体渲染着色器对象
    unsigned int shaderProgram_object = linkShaderProgram(vertexShaderSource_object,
fragmentShaderSource_object);

    // 文本渲染着色器对象
    unsigned int shaderProgram_text = linkShaderProgram(vertexShaderSource_text,
fragmentShaderSource_text);

    // 渲染文本
    glm::mat4 projection_text = glm::ortho(0.0f, static_cast<float>(SCR_WIDTH), 0.0f,
static_cast<float>(SCR_HEIGHT));
    glUseProgram(shaderProgram_text);
    glUniformMatrix4fv(glGetUniformLocation(shaderProgram_text, "projection"), 1,
GL_FALSE, glm::value_ptr(projection_text));

```



```

// 初始化FreeType, 读取字体
int retFT = initFreeType();
if (retFT == -1) return -1;

createTextData();
createObjectData();
createObjectTexture();

loadObjectTexture(shaderProgram_object);

// render loop
// -----
// glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
while (!glfwWindowShouldClose(window))
{
    // per-frame time logic
    // -----
    float currentFrame = static_cast<float>(glfwGetTime());
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    // input
    processInput(window);

    // render
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
    // glClear(GL_COLOR_BUFFER_BIT);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glUseProgram(shaderProgram_object);
    vector<string> cam_info = RenderObject(shaderProgram_object);

    glUseProgram(shaderProgram_text);
    RenderText(shaderProgram_text, cam_info[2], 25.0f, 25.0f, 0.4f, glm::vec3(0.5,
0.8f, 0.2f));
    RenderText(shaderProgram_text, cam_info[1], 25.0f, 50.0f, 0.4f, glm::vec3(0.5,
0.8f, 0.2f));
    RenderText(shaderProgram_text, cam_info[0], 25.0f, 75.0f, 0.4f, glm::vec3(0.5,
0.8f, 0.2f));
    RenderText(shaderProgram_text, "CG Lab 1 ", 540.0f, 570.0f, 0.5f,
glm::vec3(0.3, 0.7f, 0.9f));

    // glfw: swap buffers and poll IO events (keys pressed/released, mouse moved
etc.)

```

```

// -----
---
    glfwSwapBuffers(window);
    glfwPollEvents();
}

// optional: de-allocate all resources once they've outlived their purpose:
// -----
glDeleteVertexArrays(1, &VAO_object);
glDeleteBuffers(1, &VBO_object);
glDeleteBuffers(1, &EBO_object);
glDeleteProgram(shaderProgram_object);

glDeleteVertexArrays(1, &VAO_text);
glDeleteBuffers(1, &VBO_text);
glDeleteProgram(shaderProgram_text);

// glfw: terminate, clearing all previously allocated GLFW resources.
// -----
glfwTerminate();
return 0;
}

```

```

// 渲染单行文本
void RenderText(unsigned int shaderProgram_text, std::string text, float x, float y,
float scale, glm::vec3 color)
{
    glEnable(GL_CULL_FACE);
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    // activate corresponding render state
    glUniform3f(glGetUniformLocation(shaderProgram_text, "textColor"), color.x,
color.y, color.z);
    --pass--
    glBindTexture(GL_TEXTURE_2D, 0);
}

```

```

// 渲染物体
vector<string> RenderObject(unsigned int shaderProgram_object)
{
    glDisable(GL_CULL_FACE);
}

```

```
glDisable(GL_BLEND);

--pass--
//glDrawArrays(GL_TRIANGLES, 0, 6);
unsigned int modelLoc = glGetUniformLocation(shaderProgram_object, "model");
glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(model));
unsigned int projectionLoc = glGetUniformLocation(shaderProgram_object,
"projection");
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));
unsigned int viewLoc = glGetUniformLocation(shaderProgram_object, "view");
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));

--pass--

vector<string> ret;
ret.push_back(glm::to_string(cameraPos));
ret.push_back(glm::to_string(cameraPos + cameraFront));
ret.push_back(glm::to_string(cameraUp));

string cameraPos_str = glm::to_string(cameraPos);

return ret;
}
```